

# Models of Computation

## Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Finite state automata</b>                                              | <b>3</b>  |
| 1.1      | Deterministic finite-state automaton . . . . .                            | 3         |
| 1.2      | Regular languages . . . . .                                               | 3         |
| 1.2.1    | Regular expressions . . . . .                                             | 4         |
| 1.3      | Combining automata . . . . .                                              | 4         |
| 1.4      | Concatenation . . . . .                                                   | 5         |
| <b>2</b> | <b>Non-deterministic finite automaton (NFA)</b>                           | <b>5</b>  |
| 2.1      | Computation of an NFA . . . . .                                           | 7         |
| 2.2      | DFA vs NFA computation . . . . .                                          | 7         |
| 2.3      | Converting an NFA into a DFA . . . . .                                    | 7         |
| 2.4      | Closure under regular operations . . . . .                                | 9         |
| 2.4.1    | Union . . . . .                                                           | 10        |
| 2.4.2    | Concatenation . . . . .                                                   | 10        |
| 2.4.3    | Star . . . . .                                                            | 11        |
| 2.5      | Equivalence of finite automata (FA) and regular expression (RE) . . . . . | 11        |
| 2.6      | Generalised non-deterministic finite automaton (GNFA) . . . . .           | 12        |
| <b>3</b> | <b>Non-regular languages</b>                                              | <b>13</b> |
| 3.1      | Pumping lemma . . . . .                                                   | 14        |
| 3.1.1    | Proof . . . . .                                                           | 14        |
| <b>4</b> | <b>Büchi automata</b>                                                     | <b>15</b> |
| 4.1      | $\omega$ -regular languages . . . . .                                     | 16        |
| 4.1.1    | Closure under regular operations . . . . .                                | 17        |
| 4.1.2    | Limits of regular languages . . . . .                                     | 19        |
| <b>5</b> | <b>Modal logic</b>                                                        | <b>19</b> |
| 5.1      | The box operator . . . . .                                                | 19        |
| 5.1.1    | Properties . . . . .                                                      | 20        |
| 5.2      | The diamond operator . . . . .                                            | 20        |
| 5.2.1    | Properties . . . . .                                                      | 21        |
| 5.3      | Axioms of (normal) modal logic . . . . .                                  | 21        |
| 5.4      | Kripke semantics . . . . .                                                | 21        |
| 5.4.1    | Evaluating truth . . . . .                                                | 22        |
| <b>6</b> | <b>Linear temporal logic (LTL)</b>                                        | <b>23</b> |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 6.1      | Syntax of LTL . . . . .                                      | 23        |
| 6.1.1    | Formal semantics . . . . .                                   | 23        |
| 6.2      | Transition systems . . . . .                                 | 24        |
| 6.2.1    | Execution of a transition system . . . . .                   | 24        |
| <b>7</b> | <b>Turing Machines</b>                                       | <b>25</b> |
| 7.1      | Formal definition . . . . .                                  | 26        |
| 7.2      | Computation of a Turing Machine . . . . .                    | 26        |
| 7.2.1    | Configuration . . . . .                                      | 26        |
| 7.2.2    | Acceptance . . . . .                                         | 26        |
| 7.3      | Turing-Recognisable and Turing-Decidable languages . . . . . | 27        |
| 7.4      | Multitape Turing Machine . . . . .                           | 27        |
| 7.5      | Non-deterministic Turing Machine . . . . .                   | 27        |
| 7.6      | Universal Turing Machine . . . . .                           | 28        |
| 7.7      | The Halting Problem . . . . .                                | 28        |
| 7.8      | $m$ -reducibility . . . . .                                  | 28        |
| 7.8.1    | Properties of $m$ -reducibility . . . . .                    | 29        |
| 7.9      | $m$ -completeness . . . . .                                  | 29        |
| 7.9.1    | $m$ -completeness of the Halting language $H$ . . . . .      | 29        |
| 7.9.2    | $m$ -completeness of the diagonal of HALT . . . . .          | 29        |
| 7.10     | $t$ -reducibility . . . . .                                  | 30        |
| 7.11     | Computable and partially computable functions . . . . .      | 30        |
| 7.12     | Recursion theorem . . . . .                                  | 31        |
| 7.13     | Parameter theorem . . . . .                                  | 31        |
| 7.13.1   | Theorem . . . . .                                            | 31        |
| 7.13.2   | Equivalent universal-machine form . . . . .                  | 31        |
| 7.14     | Primitive recursion . . . . .                                | 31        |
| 7.14.1   | Initial functions . . . . .                                  | 31        |
| 7.14.2   | Composition and primitive recursion . . . . .                | 32        |
| 7.14.3   | Primitive recursive functions . . . . .                      | 32        |
| 7.14.4   | Minimisation . . . . .                                       | 32        |
| 7.15     | Partially recursive functions . . . . .                      | 32        |
| 7.16     | Step-counter predicate . . . . .                             | 33        |
| 7.16.1   | Definition . . . . .                                         | 33        |
| 7.16.2   | Halting . . . . .                                            | 34        |
| 7.17     | Gödel number . . . . .                                       | 34        |
| 7.17.1   | Example . . . . .                                            | 34        |
| 7.18     | Gödel's incompleteness theorems . . . . .                    | 34        |
| 7.18.1   | First incompleteness theorem . . . . .                       | 35        |
| 7.18.2   | Second incompleteness theorem . . . . .                      | 35        |
| 7.18.3   | Robinson Arithmetic ( $Q$ ) . . . . .                        | 35        |

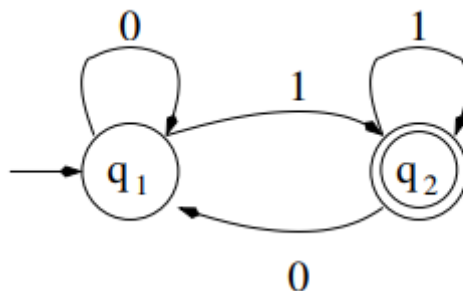
# 1 Finite state automata

## 1.1 Deterministic finite-state automaton

A deterministic finite-state automaton (DFA) is a *finite-state machine* (a mathematical model of computation) that accepts or rejects a given string of symbols by progressing through a state sequence uniquely defined by the string.

Formally, a DFA is defined as a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$  where:

- $Q$  is a finite set of states
- $\Sigma$  is a finite alphabet
- $\delta : Q \times \Sigma \rightarrow Q$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of accept states



This DFA example accepts the strings ending in 1, with:

- $Q = \{q_1, q_2\}$ ,
- $\Sigma = \{0, 1\}$ ,
- $\delta(q_1, 0) = q_1, \quad \delta(q_1, 1) = q_2, \quad \delta(q_2, 0) = q_1, \quad \delta(q_2, 1) = q_2$ ,
- $q_0 = q_1$ ,
- $F = \{q_2\}$

## 1.2 Regular languages

A language is simply a set of strings over an alphabet.

The DFA  $M$  recognises the language  $L$  if  $L = \{ w \mid M \text{ accepts } w \}$ .

- The language  $L$  is the set of all strings that the DFA  $M$  accepts

A regular language is any language that can recognised by some DFA.

- If you can build a DFA that accepts exactly the strings in that language, then that language is regular

### 1.2.1 Regular expressions

Regular expressions are another way of describing regular languages, using algebra-like rules instead of state diagrams. Regular expressions and DFAs have exactly the same expressive power.

Regular expressions are built using some basic operations, called regular operations, on languages:

- Boolean (set-theoretic) operations - general set operations can be applied to languages, since languages are sets of strings
  - Union:  $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
  - Intersection:  $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$
  - Difference:  $A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}$
  - Complement:  $\overline{A} = \Sigma^* \setminus A$
- Language theory specific operations - special to formal languages and regular expressions
  - Concatenation:  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
  - Star:  $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\} = \bigcup_{i=0}^{\infty} A^i$ 
    - \* in other words,  $A^*$  is the set of all finite strings obtained by concatenating zero or more strings from  $A$ , allowing repetition

The formal definition of a regular expression (RE) is inductive (recursive) - there are initial REs, and new REs can be obtained from old ones by means of regular operations.

$R$  is a regular expression over the alphabet  $\Sigma$  if  $R$  is any of:

1.  $a$ , for some  $a \in \Sigma$ ,
2.  $\varepsilon$  (the empty string),
3.  $\emptyset$  (the empty language),
4.  $(R_1 \cup R_2)$ , where  $R_1$  and  $R_2$  are regular expressions,
5.  $(R_1 \circ R_2)$ , where  $R_1$  and  $R_2$  are regular expressions, and
6.  $(R_1^*)$ , where  $R_1$  is a regular expression.

Note:

- the concatenation symbol can be omitted, i.e.  $R_1R_2$  means  $R_1 \circ R_2$
- parenthesis can be omitted, bearing in mind the precedence order from highest to lowest is  $^*$ ,  $\circ$ ,  $\cup$ , i.e.  $01 \cup 1^*$  means  $((01) \cup (1^*))$

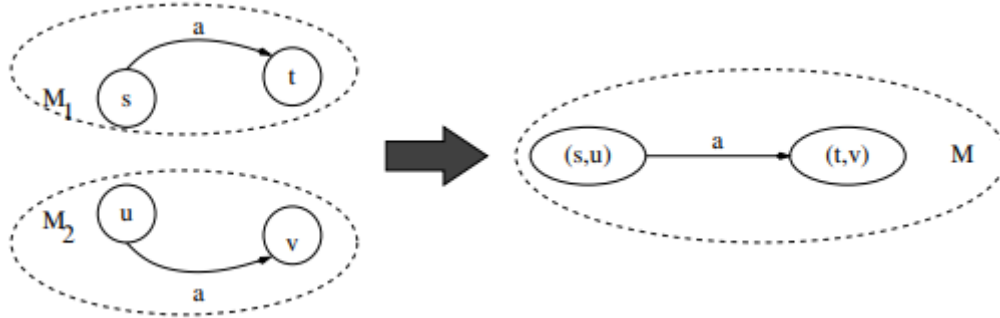
### 1.3 Combining automata

Automata can be combined in order to implement an operation on languages. This is to build a new DFA that recognises a new language, specifically one obtained by applying an operation (e.g. union) to languages that are already recognised by DFAs.

For example, given  $L_1$  is recognised by  $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  and  $L_2$  is recognised by  $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ , we can combine  $M_1$  and  $M_2$  into a new automaton  $M$  that would recognise  $L_1 \cup L_2$ .

It will not work to first simulate the input on  $M_1$ , then simulate it on  $M_2$ , and accept if either  $M_1$  or  $M_2$  or both accept, since after  $M_1$  has run on the input, the input is exhausted - A DFA cannot 'rewind' the input in order to run  $M_2$ .

The solution is to run  $M_1$  and  $M_2$  in parallel, using pairs of states (a state for each DFA). The transition function can be defined as below:



Formally, the components of the new automaton  $M = (Q, \Sigma, \delta, q_0, F)$  are:

- $Q = Q_1 \times Q_2$
- $\delta((s, u), a) = (\delta_1(s, a), \delta_2(u, a)), \quad \forall s \in Q_1, u \in Q_2, a \in \Sigma$
- $q_0 = (q_1, q_2)$
- $F = \{(s, u) \in Q \mid s \in F_1 \text{ or } u \in F_2\}$

## 1.4 Concatenation

The above method for combining automata doesn't work for recognising concatenation.

$w \in L_1 \circ L_2$  only if there are words  $x$  and  $y$  such that  $w = xy$ ,  $x \in L_1$  and  $y \in L_2$ .

If  $L_1$  is recognised by  $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  and  $L_2$  is recognised by  $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ , then to recognise  $w$ ,  $M_1$  would need to run on some prefix of  $w$ , and  $M_2$  on the rest. The problem lies in finding the break point for this - the only way to do this is to guess it non-deterministically.

In order to implement concatenation, non-deterministic finite automata are required.

## 2 Non-deterministic finite automaton (NFA)

A nondeterministic finite-state automaton (NFA) is a finite-state machine that accepts or rejects a given string of symbols by progressing through one or more possible state sequences defined by the string, where at least one such sequence leads to an accepting state.

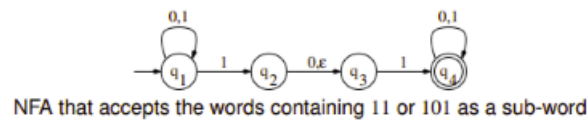
Formally, an NFA is defined as a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$  where:

- $Q$  is a finite set of states

- $\Sigma$  is a finite alphabet
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of accept states

The only difference between the DFA and NFA definitions is in the transition function:

- in a DFA, the transition function maps a state and an input symbol to exactly one next state
- in an NFA, the transition function maps a state and an input symbol (or  $\varepsilon$ ) to a set of possible next states, allowing multiple or zero transitions. Here,  $\varepsilon$  represents the empty string, meaning the automaton can move to the next state without consuming any input symbol



From this example, three potential problems can be identified:

- there is more than one next possible state (in  $q_1$ , an input 1 can lead to  $q_1$  or  $q_2$ )
- $\varepsilon$ -transition (in  $q_2$ , if the next symbol is 0, and  $\varepsilon$  is used to transition to  $q_3$ , then the 0 will not be consumed, which may lead to rejection, even though the string may contain the substring 101)
- no ‘next’ state (in  $q_3$ , an input 0 has no next state, so the computation hangs, meaning it ends before the input has been exhausted)
  - in this case, this is treated as rejection

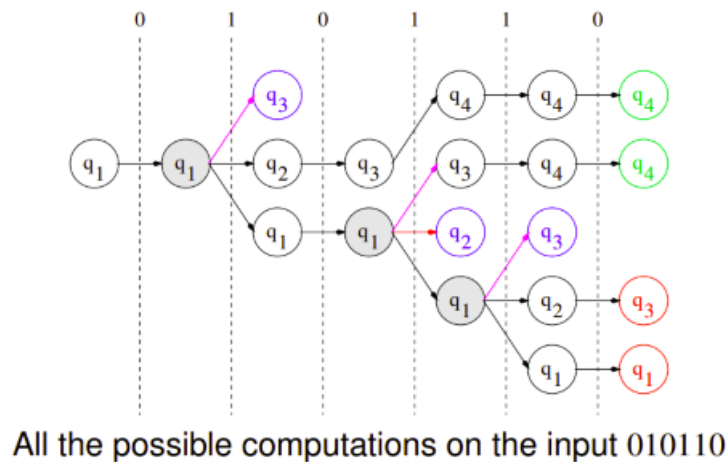
These are not actually problems, due to the nature of how NFAs work:

- an NFA can have multiple possible next states for the same input symbol and  $\varepsilon$ -transitions that move without consuming input
- conceptually, the NFA explores all possible paths simultaneously
- the input is accepted if any path reaches an accepting state after consuming the entire input
- if all paths fail, the input is rejected

Since every possibility is tried:

- multiple next states are fine - each path is explored
- $\varepsilon$ -transitions are fine - there’s also a path where the  $\varepsilon$ -transition isn’t taken, so the input isn’t lost
- paths that ‘hang’ (no next state) are fine - other valid paths will continue and may still lead to acceptance

Using the NFA example above:



## 2.1 Computation of an NFA

Let  $M = (Q, \Sigma, \delta, q_0, F)$  be an NFA and  $w = w_1 w_2 \dots w_n$  be a word over  $\Sigma \cup \{\varepsilon\}$ .

$M$  accepts  $w$  if there exists a sequence of states  $r_0, r_1, r_2, \dots, r_n$  satisfying the following conditions:

- $r_0 = q_0$
- $r_{i+1} \in \delta(r_i, w_{i+1})$  for every  $i$ ,  $0 \leq i \leq n-1$
- $r_n \in F$

## 2.2 DFA vs NFA computation

- In DFA, there is a unique computation (path). It either accepts or rejects.
- In NFA, there are, in general, many computational paths. The NFA accepts if at least one computation accepts. The NFA rejects only if all the computations reject.

### 2.3 Converting an NFA into a DFA

In an NFA, at any point, the machine could be in multiple possible states at the same time because of non-determinism. A DFA cannot be in multiple states at once - it must have exactly one current state. Therefore, in order to convert an NFA into a DFA, make each state of the DFA represent a subset of states of the NFA.

In the converted DFA:

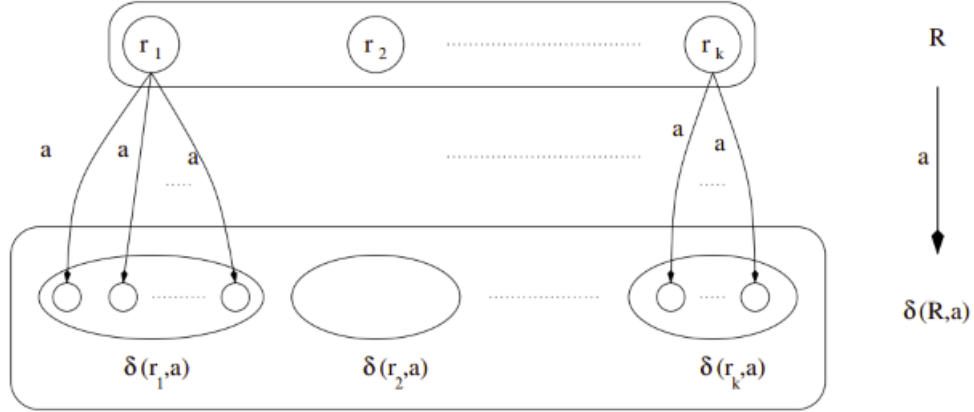
- each DFA state is a set of NFA states
- the DFA start state is the set containing the NFA start state and all states reachable from it via  $\varepsilon$ -transitions
- the DFA transition function is defined as:
  - For a DFA state  $R \subseteq Q_{NFA}$  and input symbol  $a \in \Sigma$ :
    1. Look at all NFA states in  $R$

2. For each state  $r \in R$ , see where the NFA can go on input  $a$
3. Take the union of all these resulting states
4. Add all states reachable via  $\varepsilon$ -transitions

\* This gives the next DFA state

- a DFA state is accepting if it contains any NFA accepting state

An example of how a the transition function of the DFA can be defined is given below:



More formally, let  $N = (Q, \Sigma, \delta, q_0, F)$  be an NFA recognising some language  $L$ . We construct a DFA  $M = (Q', \Sigma, \delta', q'_0, F')$  recognising the same language  $L$  as follows:

Define the  $\varepsilon$ -closure of a set of states:

$$E(R) = \{q \in Q \mid q \text{ is reachable from some } r \in R \text{ through a number of } \varepsilon\text{-transitions}\}.$$

(Note that  $R \subseteq E(R)$  because each state is reachable from itself using zero  $\varepsilon$ -transitions.)

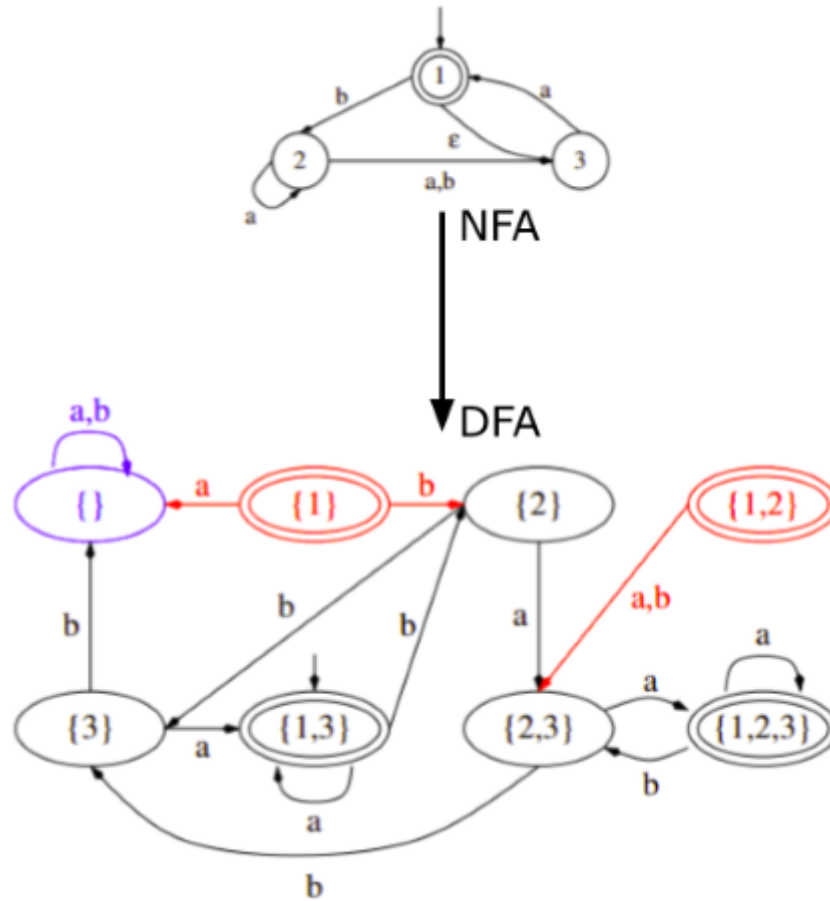
1.  $Q' = \mathcal{P}(Q)$ , the powerset (all possible subsets) of  $Q$ .
2. For  $R \in Q'$  and  $a \in \Sigma$ , define

$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ for some } r \in R\}$$

3.  $q'_0 = E(\{q_0\})$ .
4.  $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$ .

An example of the conversion of an NFA to a DFA is shown below:





Note that the red states are redundant and can be removed, as there are no ways to reach those states from the start state. The  $\{\}$  state is necessary as it collects all the paths that would ‘hang’ in the NFA.

Proving that every NFA has an equivalent DFA has the following corollary:

- A language is regular iff some NFA recognises it

## 2.4 Closure under regular operations

A class of languages (in this case the class is regular languages) is closed under an operation if applying that operation to languages in the class produces another language in the class.

Therefore, to prove the closure of the regular operations (union, concatenation and star), it must be shown that:

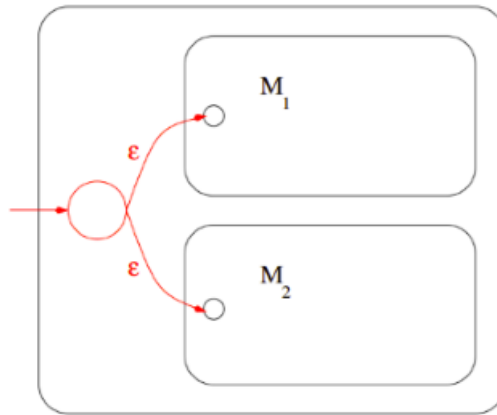
- if  $L_1$  and  $L_2$  are regular, then  $L_1 \cup L_2$ ,  $L_1 L_2$  and  $L_1^*$  are also regular

In other words, it must be proven that if  $L_1$  and  $L_2$  can be recognised by some DFA, then  $L_1 \cup L_2$ ,  $L_1 L_2$  and  $L_1^*$  can also be recognised by some DFA.

However, since we now know that every NFA has an equivalent DFA, this can be proven by proving that  $L_1 \cup L_2$ ,  $L_1 L_2$  and  $L_1^*$  can be recognised by some NFA.

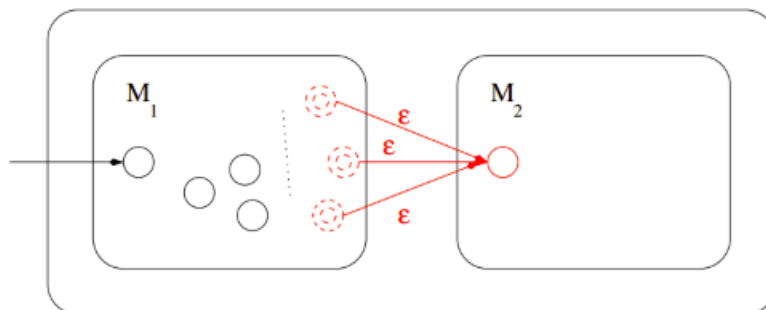
### 2.4.1 Union

1. Assume  $L_1$  and  $L_2$  are regular
2. Therefore, there exist DFAs  $M_1$  and  $M_2$  recognising them
3. Construct an NFA by:
  - adding a new start state
  - adding  $\varepsilon$  transitions from this new start state to the start states of  $M_1$  and  $M_2$
4. This NFA recognises  $L_1 \cup L_2$



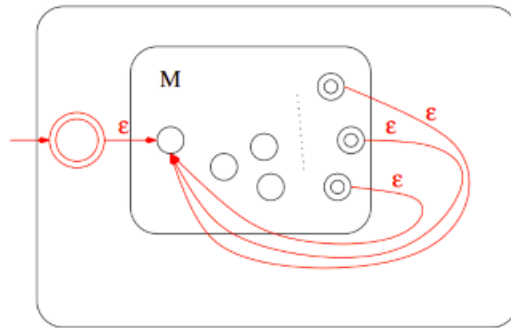
### 2.4.2 Concatenation

1. Assume  $L_1$  and  $L_2$  are regular
2. Therefore, there exist DFAs  $M_1$  and  $M_2$  recognising them
3. Construct an NFA by:
  - adding  $\varepsilon$  transitions from each accepting state of  $M_1$  to the start state of  $M_2$
  - making the accepting states only the original accepting states of  $M_2$
4. This NFA recognises  $L_1 L_2$



### 2.4.3 Star

1. Assume  $L_1$  is regular
2. Therefore, there exists a DFA  $M_1$  recognising it
3. Construct an NFA by:
  - adding a new start state which is also an accepting state
  - adding  $\varepsilon$  transitions:
    - from the new start state to the old start state
    - from each old accepting state back to the old start state
4. This NFA recognises  $L_1^*$



## 2.5 Equivalence of finite automata (FA) and regular expression (RE)

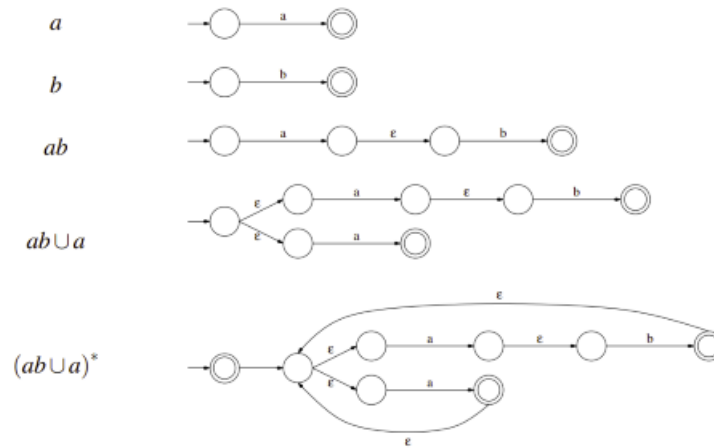
Theorem: A language is regular iff some regular expression describes it.

To prove this theorem, first prove that given a regular expression  $R$ , an FA recognising  $L(R)$  can be constructed.

This can easily be proved using structural induction:

- all initial REs can be implemented by NFAs
- every RE can be built from the initial REs using the regular operations, and we have proven that the class of regular languages is closed under regular operations

The example below shows the construction of an NFA for the regular expression  $(ab \cup a)^*$ :



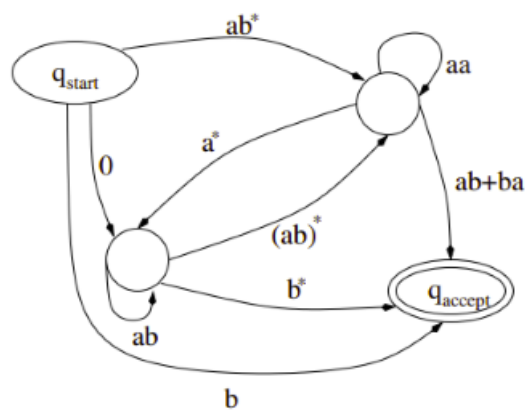
However, proving the reverse - given a DFA  $M$ , construct a RE that recognises  $L(M)$  - is harder.

## 2.6 Generalised non-deterministic finite automaton (GNFA)

A generalised non-deterministic finite automaton (GNFA) is an NFA with some additional characteristics:

- a single start state with no incoming edges
- a single accept state with no outgoing edges
- every transition is labelled with a regular expression

An example of a GNFA is shown below:

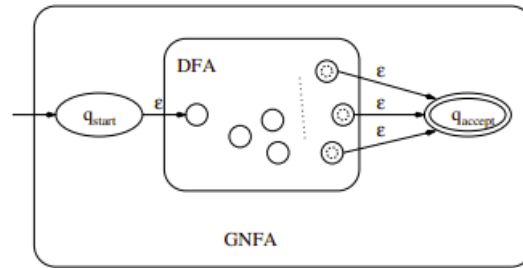


A GNFA can be used to prove  $DFA \rightarrow RE$ , since a DFA can first be converted to a GNFA, which can then be used to produce a regular expression describing the language.

To convert from a DFA to a GNFA:

1. Add a new start state  $s$ , with a  $\epsilon$  transition to the old start state
2. Add a new accept state  $f$ , with  $\epsilon$  transitions from all old accept states

3. Replace each DFA transition with a regex
4. Add missing edges and label them with  $\emptyset$



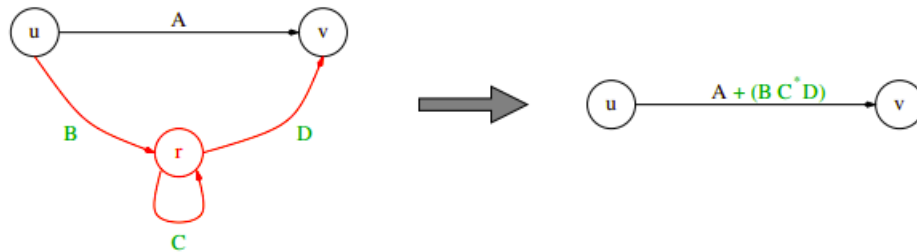
Now, to get from GNFA to a regular expression, the internal states must be eliminated one-by-one, until there is a just a single transition from the start state to the accept state, at which point the regular expression for the transition fully describes the language.

To eliminate a state  $r$ :

- For every pair of states  $u, v \neq r$ , update:

$$R_{uv} = R_{uv} \cup R_{ur}(R_{rr})^*R_{rv}$$

- Then remove  $r$  and all its edges
- Note that the case where  $u \equiv v$  must also be considered



State elimination can be used to find a single regular expression describing the set of all strings that take a finite automaton from one state to another.

### 3 Non-regular languages

A non-regular language is a language which is not recognised by an DFA.

#### PROVING A LANGUAGE NON-(OMEGA-)REGULAR IS NOT ASSESSABLE

To prove that a language  $N$  is not regular, use proof by contradiction:

1. Assume that  $N$  is regular
2. There must be some DFA that recognises  $N$
3. Use an adversary argument to show that any such DFA must fail

4. This leads to a contradiction, so the assumption was false
5. Therefore,  $N$  is not regular

The pumping lemma is a common formal adversary argument used to carry out Step 3 by exploiting the finite number of states in any DFA.

### 3.1 Pumping lemma

The pumping lemma states that:

- For every regular language  $L$ , there is a number  $p$  (the “pumping length” of  $L$ ) such that every word  $w \in L$  of length at least  $p$  can be divided into three parts,  $w = xyz$ , satisfying the following conditions:
  1.  $xy^iz \in L$  for every  $i \in \mathbb{N}$
  2.  $y$  is a non-empty string
  3. the length of  $xy$  is not greater than  $p$

Informally, this means that if  $L$  is regular, then every long string can be ‘pumped’ and the result stays in  $L$  - ‘pumped’ meaning that the string can be split into three pieces, and the middle piece repeated an arbitrary number of times.

#### 3.1.1 Proof

The pumping lemma can be proved as follows:

1. Let the DFA  $M$  recognise the language  $A$ . Let  $p$  be the number of states in  $M$ . Pick  $w \in A$  where  $|w| \geq p$
2. Since  $|w| \geq p$ , when  $w$  is input into  $M$ ,  $M$  is guaranteed to repeat a state
3. Let the first occurrence of the repeated state be  $q_r$  and the second occurrence be  $q_s$ , so  $0 \leq r < s \leq p$
4. Let:

- $x = a_1 a_2 \dots a_r$ ,
- $y = a_{r+1} \dots a_s$ ,
- $z = a_{s+1} \dots a_{|w|}$

where  $s = a_1 a_2 \dots a_{|w|}$

Then:

- $|xy| = s \leq p$
- $|y| = s - r > 0$

5. Because  $y$  corresponds to a loop:

- $\delta(q_r, y) = q_r$
- repeating  $y$  any number of times keeps the DFA in the same state  $q_s$

- so reading  $xy^iz$  always leads to the same final state as  $w$
- therefore,  $xy^iz \in L \quad \forall i \in \mathbb{N}$

Proving that a language does not satisfy the pumping lemma proves that the language is not regular, since all regular languages must satisfy it.

This is typically done by choosing a ‘bad’ input string, and pumping it up, then proving that the pumped up string is not in the language.

For example, proving that the language  $L = \{a^n b^n \mid n \geq 0\}$  is irregular:

- this language requires any number of  $a$ ’s followed by the same number of  $b$ ’s
- if  $L$  were regular, then there exists a pumping length  $p$
- any string  $s \in L$  longer than  $p$  can be written as  $s = xyz$ , with:
  - $|xy| \leq p$
  - $|y| > 0$
  - $xy^iz \in L$  for all  $i \geq 0$
- pick the string  $s = a^p b^p$
- the first  $p$  symbols of  $s$  are all  $a$ ’s, and the pumping lemma forces  $|xy| \leq p$
- therefore  $y = a^k$  for some  $k > 0$
- choose for example  $i = 0$  for the third condition of the pumping lemma
- $xy^iz = xy^0z = xz = a^{p-k} b^p$
- this string has fewer  $a$ ’s than  $b$ ’s, so  $xy^0z \notin L$
- this contradicts the pumping lemma’s third condition:  $xy^iz \in L$  for all  $i \geq 0$
- therefore, the assumption that  $L$  is regular must be false.

Note that while failing the pumping lemma proves non-regularity, satisfying it does not prove regularity.

## 4 Büchi automata

Büchi automata are a type of finite automaton for infinite strings.

They are essentially non-deterministic finite automata equipped with a special acceptance condition for infinite inputs.

We assume that the transition relation  $\Delta \subseteq Q \times \Sigma \times Q$  contains no  $\varepsilon$ -transitions; each transition consumes exactly one input symbol.

An infinite input over the alphabet  $\Sigma$  is denoted using  $\Sigma^\omega$ . A finite input over the same alphabet is denoted using  $\Sigma^*$ .

A DFA or NFA accepts a finite word  $w_1 w_2 \dots w_n \in \Sigma^*$  if there is a sequence of states  $r_0, r_1, \dots, r_n$  satisfying the following conditions:

- $r_0 = q_0$

- $(r_i, w_{i+1}, r_{i+1}) \in \Delta$  for every  $i$ ,  $0 \leq i \leq n-1$ 
  - this means that for each position in the input word, when the automaton is in state  $r_i$ , and the next input symbol is  $w_{i+1}$ , the automaton is allowed (by its transition relation  $\Delta$ ) to move to state  $r_{i+1}$ .
- $r_n \in F$

Whereas, a Büchi automaton accepts  $w_1 w_2 \dots \in \Sigma^\omega$  if there is a sequence of states  $r_0, r_1, \dots \in Q^\omega$  satisfying the following conditions:

- $r_0 = q_0$
- $(r_i, w_{i+1}, r_{i+1}) \in \Delta$  for every  $i$ ,  $i \geq 0$
- there are infinitely many  $r_i$ s in  $F$

In other words, a Büchi automaton accepts an infinite string  $w$  if and only if there exists a run of the automaton on  $w$  that visits at least one accepting state in  $F$  infinitely often.

A Büchi automaton is deterministic if it has a single initial state and exactly one outgoing transition for each state and input symbol.

#### 4.1 $\omega$ -regular languages

An  $\omega$ -regular language/expression is built upon regular languages, using the following operations:

- $A \cup B$ , where both  $A$  and  $B$  are  $\omega$ -regular
- $AB$ , where  $A$  is regular and  $B$  is  $\omega$ -regular
- $A^\omega$ , which is  $\{a_1 a_2 \dots \mid a_i \in A\}$ , i.e. an infinite sequence of words from  $A$ , where  $A$  is regular and doesn't contain the empty word

A language  $L \subseteq \Sigma^\omega$  is  $\omega$ -regular if and only if it is recognized by some non-deterministic Büchi automaton.

To prove that every language accepted by a Büchi automaton can be written using  $\omega$ -regular expressions:

1. Fix a Büchi automaton:  $\mathcal{B} = (Q, \Sigma, \Delta, q_0, F)$ 
  - a word  $\omega \in \Sigma^\omega$  is accepted by  $\mathcal{B}$  iff there exists one run  $r_0, r_1, r_2, \dots$  such that:
    - $r_0 = q_0$
    - transitions follow  $\omega$ , i.e.  $(r_i, w_{i+1}, r_{i+1}) \in \Delta$  for all  $i \geq 0$
    - some accepting state  $q \in F$  occurs infinitely often in the run
2. There exists at least one accepting state  $q \in F$  such that the run visits  $q$  infinitely many times
3. Fix that particular state  $q$ 
  - the run looks like  $q_0 \rightarrow \dots \rightarrow q \rightarrow \dots \rightarrow q \rightarrow \dots \rightarrow q \rightarrow \dots$
4. The run can be split into two parts
  - the finite prefix of the input word that takes the automaton:  $q_0 \xrightarrow{u} q$



- the set of all such finite words is exactly  $L_{q_0q}$ , which is a regular language
- An infinite sequence of finite loops from  $q$  back to  $q$ 
  - each return corresponds to a finite word that takes:  $q \xrightarrow{v} q$
  - all such words form the regular language  $L_{qq}$
  - since this happens infinitely often, the rest of the input word is  $v_1, v_2, v_3, \dots$ , with  $v_i \in L_{qq}$
  - this is exactly  $L_{qq}^\omega$
- 5. Therefore, every accepted word has the form  $uv_1v_2v_3\cdots$  with  $u \in L_{q_0q}$  and  $v_i \in L_{qq}$ 
  - the set of words accepted via this particular accepting state  $q$  is  $L_{q_0q}L_{qq}^\omega$
- 6. The repeating accepting state can be any state  $q \in F$ , so the full language accepted by the automaton is:

$$\bigcup_{q \in F} L_{q_0q}L_{qq}^\omega$$

- 7. This expression uses only regular languages, union, concatenation, and the  $\omega$ -operation. Therefore, every language accepted by a Büchi automaton is  $\omega$ -regular.

#### 4.1.1 Closure under regular operations

$\omega$ -regular languages are closed under the regular operations, suitably adapted to infinite words:

- union
  - $L_1 \cup L_2$
- intersection
  - $L_1 \cap L_2$
- complement
  - $\Sigma^\omega \setminus L$
- concatenation with regular languages
  - $R \cdot L$  where  $R \subseteq \Sigma^*$ ,  $L \subseteq \Sigma^\omega$
- $\omega$  power
  - $R^\omega$  where  $\epsilon \notin R$

#### Proofs

To prove closure under each of these operations, we must prove that given Büchi automata for the input languages, we can construct a Büchi automaton for the result under the operations.

#### Union

1. Let  $\mathcal{B}_1$  recognise  $L_1$  and  $\mathcal{B}_2$  recognise  $L_2$
2. Add a new start state

3. From this new start state, when the first input symbol is read, the automaton non-deterministically chooses to behave like  $\mathcal{B}_1$  or like  $\mathcal{B}_2$  and moves to the state that the chosen automaton would move to on that symbol
4. After this first step, the automaton continues to follow only the transitions of the chosen Büchi automaton
5. The set of accepting states of the new automaton is the union of the accepting states of  $\mathcal{B}_1$  and  $\mathcal{B}_2$
6. If, in any run, the input is accepted by the chosen automaton (i.e., the run visits accepting states infinitely often), the input word is accepted.

### Intersection

1. Let  $\mathcal{B}_1$  recognise  $L_1$  and  $\mathcal{B}_2$  recognise  $L_2$
2. Construct a new automaton whose states are all pairs of states from  $\mathcal{B}_1$  and  $\mathcal{B}_2$ 
  - that is, each state represents where we are in  $\mathcal{B}_1$  and  $\mathcal{B}_2$  simultaneously”
3. The start state of the new automaton is the pair of the start states of  $\mathcal{B}_1$  and  $\mathcal{B}_2$
4. The transitions are defined so that:
  - when reading an input symbol, the new automaton moves to the pair of successor states that each original automaton would move to on that symbol
  - in other words, it simulates both automata at the same time
5. A run is accepting if it visits an accepting state of  $\mathcal{B}_1$  infinitely often and an accepting state of  $\mathcal{B}_2$  infinitely often

**Complement** Proving the closure of  $\varepsilon$ -regular languages under complementation is beyond the scope of the module. Nice.

### Concatenation with regular languages

1. Let  $M$  be a finite automaton that recognises  $R$  and  $\mathcal{B}$  be a Büchi automaton recognising  $L$
2. From each accepting state of  $M$ , for every symbol  $a \in \Sigma$ , add transitions to the state(s) that  $B$  would move to from its start state on input  $a$
3. A word is accepted if the finite prefix is accepted by  $M$  and the remaining infinite suffix is accepted by  $B$

### $\omega$ power

1. Let  $M$  be a finite automaton that recognises  $R$
2. From each accepting state, for every symbol  $a \in \Sigma$ , add transitions to the state(s) that  $M$  would move to from its start state on input  $a$ 
  - this lets the automaton begin reading the next word in the infinite sequence without using  $\varepsilon$ -transitions.
3. Mark all accepting states of  $M$  as accepting states in the Büchi automaton

4. As there are infinitely many words in the input, the automaton will visit accepting states infinitely often, so all infinite sequences of words from  $A$  (i.e.,  $A^\omega$ ) will be accepted

#### 4.1.2 Limits of regular languages

Let  $A$  be a regular language. The limit of  $A$ ,  $\lim A$  is the language  $\{a \in \Sigma^\omega \mid a \text{ has infinitely many prefixes in } A\}$ .

This means that  $\lim A$  is the set of infinite words whose prefixes belong to  $A$  infinitely many times.

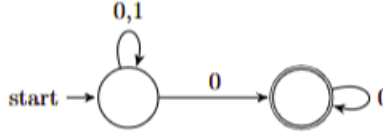
An  $\omega$ -language is a limit of a regular language if and only if some deterministic Büchi automaton recognises it.

Not every Büchi automaton can be made deterministic:

1. Let

$$L = \{a \in \{0,1\}^\omega \mid a \text{ contains finitely many 1s}\}.$$

2. There exists a nondeterministic Büchi automaton that recognises  $L$  (it nondeterministically guesses the last occurrence of 1).



3. Suppose, for contradiction, that there exists a deterministic Büchi automaton recognising  $L$ .
4. By the theorem “ $\omega$ -languages recognised by deterministic Büchi automata are exactly limits of regular languages”, we have:  $L = \lim A$  for some regular language  $A \subseteq \{0,1\}^*$
5. However,  $L$  is not a limit language, since limit languages express properties that hold infinitely often, whereas  $L$  requires that 1 occurs only finitely many times.
6. This is a contradiction. Therefore, no deterministic Büchi automaton recognises  $L$ .

## 5 Modal logic

Modal logic extends propositional logic by introducing operators that talk about modes of truth, such as:

- Necessity: something is true in all possible situations
- Possibility: something is true in at least one possible situation

Each operator’s meaning depends on the context/interpretation used.

### 5.1 The box operator

Alethic (necessity) modal logic:

- $\Box\phi$  means that  $\phi$  is true in all possible situations i.e., necessarily  $\phi$

- typically, the truth of a propositional statement is determined by examining the state of affairs in that world
- for example, if  $P$  and  $Q$  are both true in the current world,  $P \wedge Q$  will be true in this world
- if  $P$  is true in all conceivable worlds,  $\Box P$  is true — that is,  $P$  is necessarily true

Epistemic (knowledge) modal logic:

- $\Box\phi$  means that I know  $\phi$ 
  - for example:
    - \*  $P$  means that  $P$  is true
    - \*  $\Box P$  means that I know  $P$  is true

Provability (formal logic):

- $\Box\phi$  means that  $\phi$  is provable
  - for example, take the statement  $P$ , representing a complex formula
    - \*  $P$  may be true, but if we can't prove it,  $\Box P$  is false

Deontic logic:

- $\Box\phi$  means 'I must do  $\phi$ '

### 5.1.1 Properties

Being necessary (or known/provable) guarantees truth, but being true does not guarantee necessity (or knowledge/provability):

- $\Box\phi \rightarrow \phi$  is true, but  $\phi \rightarrow \Box\phi$  may not be true

If something must be true, then it must be true that it must be true:

- $\Box\phi \rightarrow \Box\Box\phi$

## 5.2 The diamond operator

Alethic (possibility) modal logic:

- $\Diamond\phi$  means that  $\phi$  is true in at least one possible situation, i.e., possibly  $\phi$ .
- Formally,  $\Diamond\phi \equiv \neg\Box\neg\phi$
- Example:
  - $P$  = It is raining.
  - $\Diamond P$  = It is possible that it is raining (true if there is at least one world where it rains)

Epistemic (knowledge) modal knowledge:

- $\Diamond\phi$  means that it is possible for  $\phi$  to be true (it is not known that is false)

Provability (formal logic):

- $\Diamond\phi$  means that  $\phi$  is possibly provable; it cannot be disproved

Deontic logic:

- $\Diamond\phi$  means that ‘I may do  $\phi$ ’

### 5.2.1 Properties

Negation duality:

- $\neg\Box\phi \equiv \Diamond\neg\phi$ 
  - ‘it is not necessary that  $\phi$ ’ is the same as ‘it is possible that  $\neg\phi$ ’
- $\neg\Diamond\phi \equiv \Box\neg\phi$ 
  - LHS means that there is not at least one possible situation where  $\phi$  is true, i.e.,  $\phi$  is impossible
  - ‘ $\phi$  is impossible’ is the same as ‘ $\neg\phi$  is necessary’

## 5.3 Axioms of (normal) modal logic

The usual rules and axioms of propositional logic hold.

Additionally, there is:

- Necessitation rule, N

$$\frac{\phi}{\Box\phi}$$

- anything that is provable is necessarily true

- Distribution axiom, K

$$\Box(\phi \rightarrow \psi) \rightarrow (\Box\phi \rightarrow \Box\psi)$$

- necessity distributes over implication
- if it is necessary that  $\phi$  implies  $\psi$ , then if  $\phi$  is necessary,  $\psi$  must also be necessary

## 5.4 Kripke semantics

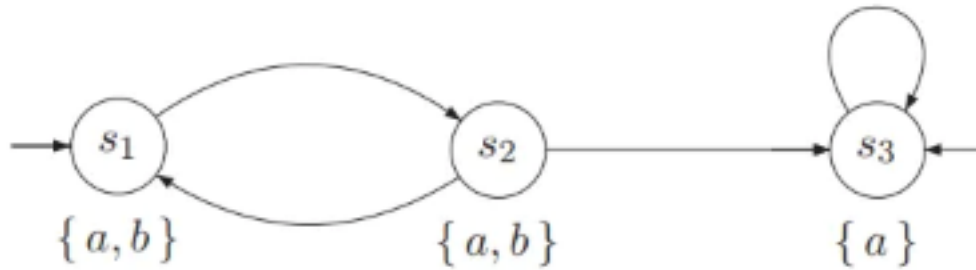
Kripke semantics gives modal logic, specifically the  $\Box$  and  $\Diamond$  operators, a precise meaning.

A Kripke model consists of:

1. Atomic propositions
  - a set  $P$  of propositional variables,  $P = \{p, q, r, \dots\}$
  - each is just a Boolean variable
2. Worlds
  - a set  $W$  of worlds (or states),  $W = \{w_1, w_2, w_3, \dots\}$
  - each world represents a possible situation
3. A valuation function

- $f : P \times W \rightarrow \{true, false\}$
  - for every proposition and every world, the function tells you whether that proposition is true in that world
4. An accessibility relation
- $R \subseteq W \times W$
  - if  $(u, v) \in R$ , it means that world  $v$  is accessible from world  $u$
  - this relation determines what worlds are considered when evaluating  $\Box$  and  $\Diamond$

Below is an example of a Kripke model.



In this model:

- $P = \{a, b\}$
- $W = \{s_1, s_2, s_3\}$
- $f : P \times W \rightarrow \{true, false\}$ :
  - $f(a, s_1) = true, \quad f(b, s_1) = true$
  - $f(a, s_2) = true, \quad f(b, s_2) = true$
  - $f(a, s_3) = true, \quad f(b, s_3) = false$
- $R = \{(s_1, s_2), (s_2, s_1), (s_2, s_3), (s_3, s_3)\}$

#### 5.4.1 Evaluating truth

$\Box a$  holds at  $u$  if and only if  $a$  holds at every  $v$ , such that  $(u, v) \in R$ .

- $a$  is necessary at  $u$  if  $a$  is true in all worlds accessible from  $u$

$\Diamond a$  holds at  $u$  if and only if  $a$  holds at some  $v$ , such that  $(u, v) \in R$ .

- $a$  is possible at  $u$  if  $a$  is true in at least one world accessible from  $u$

## 6 Linear temporal logic (LTL)

Linear temporal logic (LTL) is like modal logic, but the ‘possible worlds’ are replaced by points in a linear timeline.

The features of linear temporal logic are:

- states and atomic propositions
  - there is a set of atomic propositions, and each state (or moment in time), each proposition is either true or false
- linear timeline
  - the timeline is denoted by a sequence of states  $A_0, A_1, A_2, \dots$
  - state  $A_0$  is at time 0, i.e., the current state
  - $A_1$  is the next state, at time 1, and so on
  - propositional logic alone talks only about the current state  $A_0$
- temporal operators
  - LTL adds operators that talk about time, not just the current state:
    - \*  $\bigcirc a$ : next  $a$ .  $a$  is true in the next state
    - \*  $a \cup B$ :  $a$  until  $b$ .  $a$  is true until  $b$  becomes true

### 6.1 Syntax of LTL

We are given a finite set  $AP$  of atomic propositions (boolean variables), Boolean connectives plus two temporal modalities,  $\bigcirc$  (next) and  $\cup$  (until).

A formula in LTL is defined by the following grammar:

$$\varphi := \text{true} \mid a \mid \varphi_1 \wedge \varphi_2 \mid \neg \varphi \mid \bigcirc \varphi \mid \varphi_1 \cup \varphi_2$$

where  $a \in AP$  and  $\varphi_1, \varphi_2$  are LTL formulae.

Other modalities can be derived:

- $a$  is true in some future state:  $\Diamond a \equiv \text{true} \cup a$
- $a$  is true in all future states:  $\Box a \equiv \neg \Diamond \neg a$

#### 6.1.1 Formal semantics

A world is labelled by the  $AP$  that are true in it, so it is just a letter from the alphabet  $2^{AP}$  (the set of all subsets of  $AP$ ).

A word  $\sigma$  is an infinite sequence of worlds, i.e.  $\sigma \in (2^{AP})^\omega$ .

The satisfaction relation,  $\sigma \models \varphi$ , where  $\sigma = A_0 A_1 \dots$  is a word and  $\varphi$  is a formula, is defined recursively by

- $\sigma \models \text{true}$

- $\sigma \models a$  iff  $a \in A_0$
- $\sigma \models \varphi_1 \wedge \varphi_2$  iff  $\sigma \models \varphi_1$  and  $\sigma \models \varphi_2$
- $\sigma \models \neg\varphi$  iff  $\sigma \not\models \varphi$
- $\sigma \models \bigcirc\varphi$  iff  $A_1 \dots \models \varphi$
- $\sigma \models \varphi_1 U \varphi_2$  iff there is  $i \geq 0$  s.t.  $A_i \dots \models \varphi_2$  and  $A_j \dots \models \varphi_1$  for all  $0 \leq j < i$

The set of all words that satisfy a formula  $\varphi$  is called  $\text{Words}(\varphi)$ .

## 6.2 Transition systems

A transition system  $TS$  is a mathematical model that represents all possible executions of a system.

Formally,  $TS = (S, \rightarrow, I, AP, L)$ , where:

- $S$  is a finite set of states
  - each state represents a possible configuration of the system
- $\rightarrow \subseteq S \times S$  is the transition relation
  - this is a binary relation that defines how the system moves from one state to another
  - it is left total, meaning that for every state  $s_1 \in S$ , there exists some  $s_2 \in S$  such that  $s_1 \rightarrow s_2$ 
    - \* this ensures the system can always make a move; there are no dead ends
- $I \subseteq S$  is the set of initial states
  - it is the set of states where execution can start
- $AP$  is a finite set of atomic propositions
  - these are Boolean variables that describe properties of states
- $L : S \rightarrow 2^{AP}$  is the labelling function
  - it assigns truth values to atomic propositions at each state
  - $L(s)$  is the set of  $AP$ s that are true at state  $s$

### 6.2.1 Execution of a transition system

A run of  $TS$  is an infinite sequence of states

$$s_0 \rightarrow s_1 \rightarrow \dots$$

where  $s_0 \in I$ , which produces an infinite trace  $\sigma \in (2^{AP})^\omega$ ,  $\sigma = L(s_0)L(s_1)\dots$

The set of all possible traces of the TS is called  $\text{Traces}(TS)$ .

$TS$  satisfies  $\varphi$ ,  $TS \models \varphi$ , if  $\text{Traces}(TS) \subseteq \text{Words}(\varphi)$  i.e. if each trace of the  $TS$  satisfies the formula  $\varphi$ .

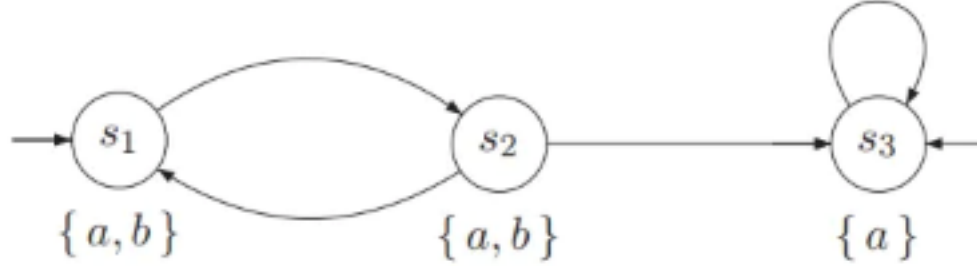
It is possible that both  $TS \not\models \varphi$  and  $TS \not\models \neg\varphi$ :



- some traces may satisfy  $\varphi$ , while others may violate  $\varphi$
- the system is non-deterministic with regards to the property.

$TS \models \varphi$  essentially means that, no matter how the system runs, the property is always true.

Applying this to the Kripke model from earlier:



$$Traces(TS) = (\{a, b\}\{a, b\})^* a^\omega \cup \{a, b\}^\omega$$

The following hold:

- $s_1 \models \bigcirc(a \wedge b)$
- $s_3 \not\models \bigcirc(a \wedge b)$
- $TS \not\models \bigcirc(a \wedge b)$
- $TS \models \Box(\neg b \rightarrow \Box(a \wedge \neg b))$

## 7 Turing Machines

A Turing machine is a formal model used to define an algorithm. Anything that can be computed by any machine can also be computed by a Turing Machine.

A Turing machine has:

- infinite ‘tape’ (memory), extending infinitely (to the right)
- a finite-state ‘program’ that controls a tape head
  - the head can read, write and move around (in both directions) on the tape

Each step of the machine follows a rule (instruction):

- look at the current state
- look at the symbol under the tape head
- based on these, perform the following actions:
  - write a symbol (possibly the same one)
  - move the head left or right
  - change to a new state

## 7.1 Formal definition

A Turing Machine (TM) is a 7-tuple  $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ , where

1.  $Q$  is the set of states,
2.  $\Sigma$  is the input alphabet not containing the special blank symbol  $\epsilon$ ,
3.  $\Gamma$  is the tape alphabet satisfying  $\Sigma \subseteq \Gamma$  and  $\epsilon \in \Gamma$ ,
4.  $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$  is the transition function
  - $\delta(q, s) = (q', s', M)$  means if the current state is  $q$  and the cell pointed by the head contains  $s$ :
    - the next state will be  $q'$
    - the current cell will be rewritten to contain  $s'$
    - the head will move to the left if  $M = L$  or to the right if  $M = R$ . If the head points to the leftmost cell and  $M = L$ , the head stays there
5.  $q_0 \in Q$  is the start state,
6.  $q_{accept} \in Q$  is the accept state, and
7.  $q_{reject} \in Q$  is the reject state,  $q_{reject} \neq q_{accept}$

## 7.2 Computation of a Turing Machine

The tape is infinite in theory (so you never run out of memory), but at any moment, only a finite portion is actually used:

- you could keep writing forever, but at any time you've only written a finite amount
- the first blank symbol marks the end of the meaningful input/data

### 7.2.1 Configuration

A configuration consists of three items: the current state, the tape content, and the head location.

- The configuration  $C_1$  *yields* the configuration  $C_2$  if the TM can legally go from  $C_1$  to  $C_2$  in a single step.
- The start configuration on an input  $w \in \Sigma^*$  consists of:
  - the start state  $q_0$ ,
  - $w$  as the tape content,
  - and the head location being the first (leftmost) position of the tape
- An accepting (rejecting) configuration is a configuration whose state is  $q_{accept}$  ( $q_{reject}$ , respectively). Accepting and rejecting configurations are halting configurations.

### 7.2.2 Acceptance

A TM  $M$  accepts an input  $w$  if there is a sequence of configurations  $C_1, C_2, \dots, C_k$  such that

1.  $C_1$  is the start configuration of  $M$  on input  $w$ ,
2.  $C_i$  yields  $C_{i+1}$  for  $1 \leq i \leq k-1$ , and
3.  $C_k$  is an accepting configuration.

The set of strings accepted by  $M$  constitutes the language of  $M$ , denoted by  $L(M)$ .

### 7.3 Turing-Recognisable and Turing-Decidable languages

A language  $L$  is Turing-Recognisable (or semi-decidable), if there is a TM  $M$  that recognises it, i.e.  $L = L(M)$ :

- There exists a Turing Machine  $M$  that accepts every string in  $L$

A language  $L$  is Turing-Decidable (or just decidable), if there is a TM  $M$  that accepts every  $w \in L$  and rejects every  $w \notin L$ .

If  $M$  recognises  $L$ , it may or may not halt on words not in  $L$ . However, if  $M$  decides  $L$ , it always halts.

### 7.4 Multitape Turing Machine

A Multitape TM is like a normal Turing Machine, but instead of one tape, it has  $k$  tapes, each with its own head.

At each step, the machine looks at:

- the current state
- the symbols on all  $k$  tapes

Then it:

- writes on each tape
- moves each head (left or right)
- changes state

Every Multitape TM has an equivalent single-tape TM:

- write the contents of all tapes on the single-tape consecutively, separated by a special character (e.g. blank space)
- double the tape alphabet, so that for every original symbol  $s$ , there is a marked version  $\dot{s}$  which indicates the position of a head
- to simulate one Multitape step:
  - Scan the tape to find:
    - \* the marked symbols (head positions)
    - \* the symbols under each head
  - figure out what the multitape machine would do
  - Scan again to:
    - \* update symbols
    - \* move the head markers

### 7.5 Non-deterministic Turing Machine

In a non-deterministic TM, for each configuration, there can be multiple possible moves.

A non-deterministic TM has transition function:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

This means that, given a state and symbol, the machine returns a set of possible actions.

The machine accepts an input if there is a computation that ends into an accepting configuration.  
Every non-deterministic TM has an equivalent deterministic TM.

## 7.6 Universal Turing Machine

Every TM  $M$  can be encoded as a word over a finite alphabet.  $\langle M \rangle$  is used to denote the encoding of a TM  $M$ .

A Universal Turing Machine is a TM  $U$  that takes a two part input the encoding of a TM  $M$ ,  $\langle M \rangle$ , and a word  $w$ , and simulates  $M$  on  $w$ .

## 7.7 The Halting Problem

The Halting Problem is as follows:

Given a TM  $M$ , and a word  $w$  does  $M$  terminate on  $w$ ?

This problem is Turing-recognisable:

- Run a Universal TM on the pair  $(\langle M \rangle, w)$
- Accept if the computation eventually terminates

The Halting Problem is not Turing-decidable:

- Assume a machine  $H$  exists that decides halting: it always says whether  $M(w)$  halts
- Build a new machine  $D(M)$ :
  - run  $H(M, M)$
  - if  $H$  says ‘halts’, then loop forever
  - if  $H$  says ‘loops’, then halt
- Run  $D(D)$
- If  $D(D)$  halts, it must loop (by definition); contradiction
- If  $D(D)$  loops, it must halt (by definition); contradiction
- So  $H$  cannot exist
- Therefore the Halting Problem is undecidable (not Turing-decidable)

## 7.8 $m$ -reducibility

$m$ -reducibility (many-one reducibility) is a way to compare the difficulty of decision problems (languages).

A language  $A$  is many-one reducible to a language  $B$ , written

$$A \leq B$$

if:

- there exists a computable function  $f$  (equivalently, a Turing machine  $F$  that halts on all inputs), such that for every input  $x$ :

$$x \in A \iff f(x) \in B$$

This means that  $A$  can be solved by:

- transforming an instance  $x$  into  $f(x)$
- then solving  $B$  on  $f(x)$

Therefore,  $B$  is at least as hard as  $A$ .

### 7.8.1 Properties of $m$ -reducibility

If  $A \leq B$ , then:

- if  $B$  is Turing decidable, so is  $A$
- if  $B$  is Turing recognisable, so is  $A$
- if  $A \leq B$ , and  $B \leq C$ , then  $A \leq C$
- if  $A \leq B$ , and  $B \leq A$ , then  $A \equiv B$ 
  - $A$  and  $B$  are equally difficult

## 7.9 $m$ -completeness

A language  $A$  is  $m$ -complete if:

- $A$  is Turing recognisable, and
- for every Turing recognisable language  $B$ ,  $B \leq A$

Informally, if  $A$  is  $m$ -complete, then  $A$  is as hard as any other Turing-recognisable language.

Furthermore:

- if  $A$  is  $m$ -complete and  $A \leq B$  and  $B$  is Turing-recognisable, then:
  - $B$  is  $m$ -complete

### 7.9.1 $m$ -completeness of the Halting language $H$

The Halting language  $H$  is defined by

$$HALT = \{\langle M, w \rangle \mid M \text{ halts on input } w\}$$

That is, the set of all machine-input pairs, such that:

- when you run  $M$  on  $w$ ,
- $M$  eventually halts (and either accepts or rejects)

$H$  is  $m$ -complete:

- pick any Turing recognisable language  $A$
- $A$  is recognised by some TM  $M_A$
- define a reduction  $f(w) = \langle M_A, w \rangle$
- $f$  is computable and  $w \in A \iff \langle M_A, w \rangle \in HALT$

### 7.9.2 $m$ -completeness of the diagonal of $HALT$

$H_0$  is the diagonal of  $H$ :

- this is the language  $\{\langle M \rangle \mid M \text{ terminates on } \langle M \rangle\}$
- i.e., the set of Turing machines that halt when given their own encoding as input

$H_0$  is  $m$ -complete:

- given a word  $\langle M, w \rangle$
- crate a TM  $N_{M,w}$  that simulates  $M$  on  $w$ 
  - this TM ignores the input
  - this can be done using a universal Turing machine
- $N_{M,w}$  terminates on any input if and only if  $M$  terminates on  $w$
- in particular,  $N_{M,w}$  terminates on  $\langle M, w \rangle$  if and only if  $M$  terminates on  $w$

## 7.10 $t$ -reducibility

An oracle for a language  $A$  is a black-box that takes a word  $w$  as an input, and instantly and correctly determines whether  $w \in A$ . Essentially, an oracle for a language instantly decides that language.

An oracle Turing machine, denoted  $M^A$ , is a Turing machine that has the additional capability of making queries to an oracle for the language  $A$ .

A language  $A$  is  $t$ -reducible to a language  $B$ , denoted

$$A \leq_t B$$

if  $A$  is decidable by some oracle Turing machine  $M^B$ .

Informally,  $A \leq_t B$  means that  $A$  can be decided using an oracle for  $B$ .

If  $A \leq_t B$ , and  $B$  is Turing-decidable, then  $A$  is Turing-decidable.

Every many-one reduction is a Turing reduction:

$$A \leq_m B \implies A \leq_t B$$

The converse is not necessarily true.

## 7.11 Computable and partially computable functions

A total function

$$f : \Sigma^* \rightarrow \Sigma^*$$

is computable if there is a TM  $F$  such that for every input  $x \in \Sigma^*$ ,  $F$  halts and produces  $f(x)$  as output.

A partial function

$$g : \Sigma^* \rightarrow \Sigma^*$$

is partially computable if there is a TM  $G$  such that:

- on any input  $x \in \text{dom}(g)$ ,  $G$  halts and produces  $g(x)$  as output
- on any input  $x \notin \text{dom}(g)$ ,  $G$  does not halt

Using these definitions, the following are equivalent for a language  $L$ :

- $L$  is Turing-recognisable
- $L$  is the domain of a partially computable function
- $L$  is the range of a computable function
- $L$  is the range of a partially computable function

## 7.12 Recursion theorem

Let  $F(y, x)$  be a Turing machine that takes two inputs.

Then there exists a Turing machine  $R(x)$  such that for every input  $x$ :

$$R(x) = F(\langle R \rangle, x)$$

This means that there exists a machine  $R$  that can use its own description as part of its computation.

In other words:

- $R$  can access its own encoding  $\langle R \rangle$
- and behave exactly like  $F$  when given that encoding as input

This means that:

- A machine can ‘know’ its own code
- We can construct programs that refer to themselves
- This enables self-referential computation

## 7.13 Parameter theorem

### 7.13.1 Theorem

Let  $M(y, x)$  be a Turing machine with two inputs.

Then there exists a Turing machine constructor  $S(\cdot, \cdot)$  such that:

- given inputs  $\langle M \rangle$  and  $y$ ,
- it produces a machine  $\langle M_y \rangle$ ,

where  $M_y$  is the machine  $M$  with parameter  $y$  fixed (hard-coded), and for all  $x$ :

$$M_y(x) = M(y, x)$$

### 7.13.2 Equivalent universal-machine form

Let  $U$  be a Universal Turing Machine. Then:

$$U(S(\langle M \rangle, y), x) = M(y, x)$$

This means that

- We can ‘specialise’ a TM by fixing one of its inputs in advance
- This produces a new TM with one fewer input
- The fixed value is built into the machine’s code

## 7.14 Primitive recursion

### 7.14.1 Initial functions

Consider these initial functions, defined on the natural numbers:

- $s(x) = x + 1$ 
  - successor (adds 1)
- $n(x) = 0$ 
  - zero function
- $u_i^n(x_1, x_2, \dots, x_n) = x_i$ 
  - the projections (select an input)

### 7.14.2 Composition and primitive recursion

From these initial functions we can construct new ones using:

- composition
  - combine functions by feeding outputs into other functions
- primitive recursion
  - defines functions using a\*bounded loop
  - for example: addition by repeated increment
    - \*  $x + 0 = x$
    - \*  $x + (n + 1) = (x + n) + 1$

In these functions:

- the number of steps is fixed by the input
- so computation always terminates

### 7.14.3 Primitive recursive functions

Functions obtained using only:

- initial functions
- composition
- primitive recursion

are called primitive recursive functions:

- they are total functions (always halt)
- they cannot express unbounded search or non-termination

So they cannot model general computation involving potential non-termination.

### 7.14.4 Minimisation

To extend expressiveness, we add minimisation.

Define a function by:

- search for the smallest  $y$  such that  $f(x, y) = 0$

Formally: - try  $y = 0, 1, 2, 3, \dots$  - return the first  $y$  that works

Then: - If such a  $y$  exists, then return it - If not, then the computation runs forever (undefined)

## 7.15 Partially recursive functions

Functions obtained using: - initial functions - composition - primitive recursion - minimisation



are called partially recursive functions.

Partially recursive functions = Turing-computable partial functions

## 7.16 Step-counter predicate

The step-counter predicate is used to formalise the question:

- What configuration is a Turing machine in after exactly  $t$  steps?

It turns Turing machine execution into a purely mathematical function on numbers.

Rather than running a machine step-by-step, a function

$$SC([M], [w], t)$$

is defined, which directly gives the state of the machine after  $t$  steps.

- $[M]$  is the encoding of the TM  $M$
- $[w]$  is the encoding of the input string  $w$
- $t$  is the number of computation steps

Therefore,

$$SC([M], [w], t)$$

means

- simulate machine  $M$  on input  $w$  for exactly  $t$  steps

The output is

$$[q, i, w']$$

which is the full configuration of the machine after  $t$  steps:

- $q$ : current state
- $i$ : head position
- $w'$ : tape contents

### 7.16.1 Definition

The  $SC$  function is defined recursively:

Base case:

$$SC([M], [w], 0) = \text{initial configuration of } M \text{ on } w$$

Step case:

$$SC([M], [w], t + 1) = \text{Step}(SC([M], [w], t))$$

meaning:

- apply one transition of the machine each time

### 7.16.2 Halting

Halting can then be defined using  $SC$ :

- a machine halts if there exists such a  $t$  such that  $SC$  reaches a halting configuration

### 7.17 Gödel number

A Gödel number is a way of turning a mathematical object (like a sequence, string or TM) into a single natural number.

It uses the idea that everything can be encoded as a number.

For example, given a sequence of natural numbers

$$(x_1, x_2, \dots, x_n)$$

its Gödel encoding is

$$[x_1, \dots, x_n] = p_1^{x_1} \cdot p_2^{x_2} \cdots p_n^{x_n}$$

where

- $p_i$  is the  $i$ -th prime number.

Because every natural number has a unique prime factorisation:

- each sequence corresponds to exactly one number
- and each number corresponds to one sequence

#### 7.17.1 Example

For the sequence

$$(2, 1, 0)$$

the Gödel number is

$$2^0 \cdot 3^0 \cdot 5^0 = 4 \cdot 3 \cdot 1 = 12$$

### 7.18 Gödel's incompleteness theorems

Gödel incompleteness theorems are results in mathematical logic that show a fundamental limitation of formal mathematical systems.

They say that:

- any sufficiently powerful and consistent formal system cannot prove all true statements about arithmetic

### 7.18.1 First incompleteness theorem

In any formal system that:

- can express basic arithmetic, and
- is consistent (does not prove contradictions),

there exist statements that are:

- true, but
- not provable within the system

Therefore, there will always be true arithmetic statements you cannot prove:

- so the system is incomplete

**Gödel's self-referential sentence** Construct a statement that says:

- “this statement is not provable in this system”

Then either:

- the system proves it; contradiction
- the system doesn't prove it; true, but unprovable

### 7.18.2 Second incompleteness theorem

A consistent formal system strong enough for arithmetic:

- cannot prove its own consistency

### 7.18.3 Robinson Arithmetic ( $Q$ )

Robinson Arithmetic is a very weak formal system of arithmetic that is still strong enough to encode computation. Gödel's incompleteness theorem already holds for  $Q$ .

It has the following axioms:

1.  $S(x) \neq 0$
2.  $(S(x) = S(y)) \rightarrow x = y$
3.  $y = 0 \vee \exists x (S(x) = y)$
4.  $x + 0 = x$
5.  $x + S(y) = S(x + y)$
6.  $x \times 0 = 0$
7.  $x \times S(y) = x \times y + x$